温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01
Python Programming for Business

Working with JSON & Semi-Structured Data

Dawei Chen

Week 9-2, 23 April 2026

Where We Are

Date	Topic	Status
4/16	GitHub Literacy & Big Picture	✓ Done
4/21	API-Based Data Collection	✓ Done
4/23	**Working with JSON & Semi-Structured Data**	◀ **Today**
4/28	HTML Scraping & Unstructured Data	
4/30	*Research Workshop III*	

Assignment 3 (data collection report) due May 10, 23:59

Learning Objectives

By the end of this session, you should be able to:

- **Run** and **modify** the AI-generated Jupyter notebook from Tuesday
- Understand what ***JSON*** is and how it differs from tabular (CSV) data
- Parse nested JSON: access simple values, nested objects, and arrays
- Handle missing/null values in JSON safely
- Flatten JSON data into a clean tabular DataFrame for analysis

Recap: What We Did on Tuesday

On Tuesday, we:

- Used the 4D framework to scope a data collection task (AI skills repos)
- **Briefly walked through** an AI-generated Jupyter notebook that calls the GitHub API

But we ran out of time for

- Keywords, fields, pagination, and deduplication
- Hands-on practices

Quick check: How many of you ran the notebook after class? Any issues?

Reflection on Using AI

Discussion

- What have you learned from using AI in this course, from general frameworks to specific techniques?
- Has this course changed how you think about, think with AI?
- What difficulties or frustrations have you encountered?
- What do you still need to learn to use AI more effectively?

Takeaway

- Learning how to use AI is important.
- More importantly, you must be able to evaluate whether the output is accurate, useful, and appropriate.
- That is why each topic begins with a conceptual foundation before applying AI.
- Do not outsource all cognitive work to AI.

Why AI Generates Code, Not Data

<u>Why not just ask AI:</u> "Give me a CSV of AI skills repos from GitHub"?

1. **AI can't access websites or APIs directly**
   It generates text, not actions. Only your computer can call APIs and download data.
2. **Code is reusable and modifiable**
   A CSV is a one-time snapshot. Code can be re-run with different keywords or fields.
3. **Code is transparent and auditable**
   You see exactly what it does. A CSV from AI? You have no idea how it was made.
4. **You control the execution**
   You decide when to run, how much to collect, and where to save.

AI Hallucination

Reading AI-Generated Code — Your Job

Beyond “the code works”

When AI generates Python codes for you, your job is to:

- **Read** — each part and understand what it does (not memorize syntax)
- **Check** — whether it collects the data you actually need (Discernment)
- **Modify** — it when your needs change (different keywords, fields, sources)
- **Debug** — when something goes wrong (read error messages, inspect outputs)

Hands-on:
The API Collection Notebook

Run it, modify it, understand it

Guided Walkthrough: Setup (Cells 1–3)

Cell 1: Import libraries

requests (HTTP calls), pandas (tables), time (delays), json (inspect data)

Cell 2: Authentication

Paste your token, run the verification. You should see "✅ Token is valid!"

Cell 3: Search scope

Keywords list, API URL, pagination settings. This is what you'll modify later.

⚠️ **Pause here** — make sure everyone's token works before continuing.

Guided Walkthrough: API Call & Parsing (Cells 4–6)

Cell 4: Test API call

One request, 5 results. Check the status code (should be 200) and total_count.

Cell 5: Inspect raw JSON

What does the API actually return? Notice: nested objects (owner, license) and arrays (topics). We'll study this structure in Part 2.

Cell 6: Extraction function

How we pick the fields we want. Notice: .get() for safe access, special handling for nested fields and null values.

The nested structure and null values you see here — that's exactly what Part 2 teaches you to handle.

Hands-on Tasks (~15 min)

Task 1: Run the full notebook (Cells 1–10). Inspect the output CSV.

Task 2: Change the keywords in Cell 3 to a different topic. Re-run Cells 3–10.

Task 3: Add a new field in Cell 6 (e.g., “size”, “has_wiki”). Re-run Cells 6–10.

Common Issues:

- 401/403 error → check your token
- Empty results → check keyword spelling
- KeyError → field is null for some repos

Stuck? Raise your hand.
That’s what class time is for.

Debrief: What Did You Discover?

Quick Discussion:

- Did your keyword changes produce interesting results?
- Did anyone encounter a field that was missing or null for some repos?
- Did anyone get an error they couldn’t solve?

Transition: that “null” issue, and the nested structure you saw in Cell 5 — that’s exactly what we’ll learn to handle now.

Understanding JSON & Semi-Structured Data

The raw material between the API and your spreadsheet

What Is JSON?

JSON = JavaScript Object Notation

- A lightweight format for storing and exchanging data
- Used by almost every web API to send data back to you
- Human-readable (you can open it in a text editor) but not a spreadsheet
- **Very similar to Python dictionaries** — you already know the basics!

Two building blocks:

Objects { } — key-value pairs
(like Python dictionaries)

Arrays [ ] — ordered lists
(like Python lists)

Anatomy of a JSON Response

https://api.github.com/search/repositories?q=AI-skills&sort=stars&per_page=10

```
{ "total_count": 2456,                    ← Level 1: top-level object
  "items": [                              ← Level 1: array of repos
    {
      "full_name": "anthropics/...",      ← Level 2: simple value
      "stargazers_count": 8200,           ← Level 2: simple value
      "owner": {                          ← Level 2: nested object
        "login": "anthropics",            ← Level 3: value inside object
        "type": "Organization"
      },
      "topics": ["ai", "skills"],         ← Level 2: array
      "license": null                     ← Level 2: null value
    }, ...
  ]
}
```

Three levels: top-level object → array of items → fields (simple, nested, or null)

JSON vs. Tabular Data

Tabular (CSV / spreadsheet)

- Flat: every row has the same columns
- One value per cell
- Ready for analysis

JSON (API response)

- Hierarchical / nested
- Values can be other objects or arrays
- Needs parsing to become a table

Example: a repository's owner

In CSV: owner = "anthropic" (one cell, one value)

In JSON: "owner": { "login": "anthropic", "type": "Organization", "id": 123 }

The challenge: how do you turn nested JSON into flat rows for analysis?

Three Types of JSON Values

Simple values

Just use them directly

- `"stargazers_count": 8200`
- `repo["stargazers_count"]  →  8200`

Nested objects

Drill down with chained [ ]

- `"owner": { "login": "anthropics" }`
- `repo["owner"]["login"]  →  "anthropics"`

Arrays

Join, count, or check for values

- `"topics": ["ai", "skills", "claude"]`
- `", ".join(repo["topics"])  →  "ai, skills, claude"`

Null values: "license": null → if you try repo["license"]["name"], you get a **TypeError**. Use .get() or check for None first.

Why This Matters for Your Project

- Every API you'll work with returns JSON — not just GitHub
- Your job is always the same: *raw JSON → identify fields → flatten → CSV*
- The extraction function from Tuesday (Cell 6) is doing exactly this
- For your own project: the JSON structure will differ, but the process is identical
- AI can help you write the extraction code — but you need to understand the structure to tell AI what to extract

Hands-on:
Parsing JSON in Python

Let’s practice the core operations

Practice 1: Accessing JSON Values

Using a sample repo from our API response:

```
# Simple value
repo["stargazers_count"]                # → 8200

# Nested object
repo["owner"]["login"]                  # → "anthropics"
repo["owner"]["type"]                   # → "Organization"

# Array
repo["topics"]                           # → ["ai", "skills", "claude"]
len(repo["topics"])                      # → 3
", ".join(repo["topics"])                # → "ai, skills, claude"
```

Type along in VS Code. Run each line and check the output.

Practice 2: Handling Missing Values

What happens when a value is null (None)?

```
# CRASH: license is None for some repos
repo["license"]["name"]      # → TypeError!

# SAFE: use .get() with a default
repo.get("description", "")  # → "" if description is missing

# SAFE: check for None before drilling down
repo["license"]["name"] if repo.get("license") else "No license"

# This is exactly what Cell 6 does:
"license_name": repo["license"]["name"] if repo.get("license") else ""
```

In a dataset of 200+ repos, one null can crash your entire collection. Always handle it.

Practice 3: JSON → DataFrame

Turning a list of JSON objects into a Pandas DataFrame:

Method 1: Extraction function (what our notebook does)

```
rows = [extract_repo_info(repo) for repo in data["items"]]
df = pd.DataFrame(rows)
```

Method 2: pd.json_normalize() (Pandas shortcut)

```
df = pd.json_normalize(data["items"])
# Automatically flattens nested fields: owner.login, license.name, etc.
```

- **Method 1** gives you full control (choose fields, handle nulls).
- **Method 2** is faster for exploration.

What We Covered Today

Part 1: Hands-on with the API notebook

- Ran the notebook, modified keywords and fields, debugged issues

Part 2: JSON fundamentals

- Structure (objects, arrays, nesting), null handling, flattening to tabular

Key Takeaway:

- JSON is the bridge between the API and your spreadsheet.
- Understanding its structure lets you extract exactly the data you need — and tell AI exactly what to extract for you.

Next Session: HTML Scraping (4/28)

Not all data is available via APIs. Sometimes you need to read the webpage itself.

- HTML structure: tags, attributes, CSS selectors
- requests + BeautifulSoup: fetch a page and extract data
- Scraping unstructured text (e.g., README content, descriptions)
- Combining structured (API) and unstructured (scraped) data

Before next session:

- Start thinking about whether your research data source has an API or requires scraping (or both).
- Bring questions to Research Workshop III (4/30), if any.

Q&A

dchen@kean.edu CBPM B214

Office hours: Tue & Thu 11:15–12:30 | Wed 14:00–16:30